Text Sentiment Classification Using Deep Learning (RNN/LSTM)

1. Introduction

Sentiment analysis is a Natural Language Processing (NLP) task that involves determining the emotional tone behind textual data. It is widely used in applications such as product reviews, social media analysis, and customer feedback systems.

In this experiment, deep learning models including Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks are implemented to classify text sentiment as positive or negative. The goal is to compare their performance and understand how sequence modeling impacts results.

---

2. Dataset Description

The dataset used is the **IMDB Movie Reviews Dataset**, which contains 50,000 movie reviews labeled as positive or negative.

- Total samples: 50,000
- Training samples: 25,000
- Testing samples: 25,000
- Classes: Binary (Positive / Negative)

---

3. Data Preprocessing

To prepare the text data for deep learning models, the following steps were performed:

3.1 Tokenization

Each review is converted into a sequence of integers where each integer represents a word index.

Example:

- "This movie is great" → [45, 12, 78, 230]

3.2 Vocabulary Creation

Only the top 10,000 most frequent words were considered to limit complexity.

3.3 Sequence Padding

Since reviews vary in length, all sequences were padded to a fixed length of 200 words.

3.4 Final Input Format

- Input: Fixed-length numerical sequences
- Output: Binary labels (0 or 1)

---

4. Model Architectures

4.1 RNN Model

The baseline model uses a Simple RNN layer.

Architecture:

- Embedding Layer (64 dimensions)
- SimpleRNN Layer (64 units)
- Dense Output Layer (Sigmoid activation)

4.2 LSTM Model

LSTM improves upon RNN by handling long-term dependencies.

Architecture:

- Embedding Layer (128 dimensions)
- LSTM Layer (64 units)
- Dense Output Layer (Sigmoid activation)

4.3 GRU Model (Optional)

GRU is a simplified version of LSTM.

Architecture:

- Embedding Layer
- GRU Layer
- Dense Layer

---

5. Training Details

- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Batch Size: 64
- Epochs: 5
- Validation Split: 20%

---

6. Results and Analysis

6.1 Accuracy Comparison

- RNN achieves moderate accuracy but struggles with long dependencies.

- LSTM shows improved performance due to better memory handling.

6.2 Loss Analysis

- RNN shows higher validation loss (overfitting risk).
- LSTM converges more smoothly.

---

7. Evaluation Metrics

The models were evaluated using:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-score**

Observations:

- LSTM achieves higher precision and recall.
- RNN performance drops for longer sequences.

---

8. Experimentation

8.1 Sequence Length

- Short sequences → Faster training but lower accuracy
- Longer sequences → Better context but slower

8.2 Embedding Dimensions

- Low dimensions (32) → Less expressive
- Higher dimensions (128) → Better feature representation

---

9. Comparison: RNN vs LSTM

Feature	RNN	LSTM
Memory	Short-term	Long-term
Gradient Problem	Yes	No
Performance	Lower	Higher
Complexity	Simple	Complex

---

10. Why LSTM Performs Better

LSTM uses three gates:

- Input Gate → Controls new information
- Forget Gate → Removes irrelevant data
- Output Gate → Controls output

This allows LSTM to retain important information over long sequences, solving the vanishing gradient problem present in RNNs.

---

11. Conclusion

In this experiment, RNN and LSTM models were implemented for sentiment classification. The results show that LSTM significantly outperforms RNN due to its ability to capture long-term dependencies in text data.

Thus, LSTM is more suitable for NLP tasks involving sequential data such as sentiment analysis.

---

12. References

- TensorFlow/Keras Documentation
- IMDB Dataset
- Deep Learning Textbooks

---

13. Appendix (Code)

The implementation includes:

- Data preprocessing
- Model building (RNN, LSTM, GRU)
- Training and evaluation
- Graph plotting

(Code submitted separately as Jupyter Notebook)